SE212 Database Design and Database System

Term Project Final Report

Apex Youth Football Academy

Football Academy Management System (FAMS)

Team members

Noppajit Jadsorn — 682115020

Jakkaphat Didsabanjong — 682115008

Instructor: Aj. Pree

14 October 2026

Contents

1. Introduction and Background of the Business
2. Motivation
3. Business Rules
4. ERD and Tables
5. Normalization Steps and Functional Dependencies
6. Mapping (EER to Relational Schema)
7. DDL and DML
8. Database System Architecture
9. Tech Stack
10. Milestones
11. User Interface

1. Introduction and Background of the Business

Apex Youth Football Academy is a fictional youth football academy designed around **118 players, 8 coaches, 12 classes, 4 age groups**, and 36 training sessions per week. These figures are planning assumptions. The academy manages player development, coaches, training programs, registrations, attendance, and guardian information through the **Football Academy Management System (FAMS),** a web-based application supported by a centralized MySQL database.

Business background

The academy's assumed manual process uses spreadsheets, registration forms, and attendance sheets, creating risks of duplicate registrations, outdated age-group assignments, and slow attendance reporting. For example, when a player advances from Under-12 to Under-14, their registration may remain linked to an Under-12 class, resulting in an incorrect class roster. This is an illustrative scenario, not a verified incident.

FAMS addresses these problems by enforcing unique registrations, validating age-group eligibility, and connecting attendance to training sessions. It also allows staff to review a player's attendance rate and age-group history before considering promotion.

Actor	How often	Responsibilities
Administrator	Daily	Manage players, guardians, coaches, classes, programs, schedules, and registrations.
Coach	Each session	View assigned classes, record attendance, and review player participation.
Parent / Guardian	As needed	Provide contact information to academy staff for entry or updates.
Academy Manager	Weekly	Review enrollment, attendance, and player development records.

Scope

- **In scope:** Player and guardian management, coaches, age groups, primary positions, training programs, classes, schedules, registration validation, attendance, age-group history, authentication, and basic reports.
- **Out of scope:** Payments, native mobile apps, multiple branches, advanced performance analytics, and secondary playing positions.

2. Motivation

We selected this project because football academy operations require accurate relationships between players, classes, coaches, registrations, and attendance. FAMS uses **primary and foreign keys, UNIQUE constraints, and validation rules** to reduce inconsistent data, prevent duplicate records, and verify player eligibility.

Our measurable objectives are to: **(1) prevent 100% of duplicate registrations and attendance entries in constraint tests; (2) generate an individual attendance report in under 5 seconds; and (3) produce enrollment**

summaries for all 12 classes in under 5 seconds. These are targets for testing, not verified results; no unmeasured manual reporting baseline is claimed.

By centralizing records, FAMS helps staff manage daily operations efficiently and review attendance and age-group history before making player development decisions.

3. Business Rules

Business rules are enforced through database constraints and triggers where implemented, with additional validation and access control handled by the Node.js/Express application through its routes and middleware. The system uses MySQL to maintain relational data integrity.

#	Business rule	Enforced by
Players, age groups and positions		
1	An Age Group contains many Players; each Player belongs to exactly one current Age Group.	FK player.age_group_id NOT NULL
2	An Age Group can have many Classes; each Class belongs to exactly one Age Group.	FK class.age_group_id NOT NULL
3	Position: Each player has one position; a position may be shared by many players.	FK player.position_id NOT NULL
4	A Position is either Goalkeeper or Outfield Player, but not both. Every Position must belong to one subtype.	EER total/disjoint specialization; additional database or application validation required
5	An Age Group must have a non-negative minimum age and a maximum age greater than its minimum.	CHECK on age_group
6	A Player's date of birth must be valid and must not be in the future.	Application validation; database trigger where implemented
Coaches, programs and training sessions		
7	A Coach may teach many Classes; each Class must have exactly one Coach.	FK class.coach_id NOT NULL
8	A Training Program may be used by many Classes; each Class must follow exactly one Program.	FK class.program_id NOT NULL
9	A Class may contain many Training Sessions; each Session belongs to exactly one Class.	FK training_session.class_id NOT NULL
10	Each Training Session must have a valid date and an end time later than its start time.	CHECK on session times; date validation
Parents and guardians		
11	A Parent/Guardian may be linked to multiple Players, and a Player may have multiple guardians.	Junction table parent_player; composite PK and FKs
12	Every Player must have at least one guardian before completing registration.	Registration validation; service layer or trigger
13	A Player may have at most one primary guardian.	UNIQUE constraint on primary guardian per player
Player Registration and Class Enrollment		
14	A Player may register for many Classes, and a Class may have many Players.	registration table; FKs to Player and Class
15	A Player may register for the same Class only once.	UNIQUE (player_id, class_id)
16	A Player must meet the Class's Age Group requirements when registering.	Registration validation; trigger or service layer
17	Every Registration must have a registration date and valid status.	NOT NULL; status validation
Attendance		
18	A Player may have many Attendance records, and each Training Session may have many Attendance records.	attendance table; FKs to Player and Session
19	A Player may have only one Attendance record per Training Session.	UNIQUE (player_id, session_id)
20	Attendance may be recorded only for a Player with a valid registration in the Session's Class.	Attendance validation; trigger or service layer
21	Attendance status must be Present, Absent, Late, or Excused.	CHECK on attendance.status
Player age-group history		

#	Business rule	Enforced by
22	Each Player may have many Age Group History records; each record belongs to exactly one Player and one Age Group.	FKs in player_age_group_history
23	Every history record must have a start date. Its end date may be NULL but cannot precede the start date.	NOT NULL; CHECK on date range
Data integrity and reporting		
24	Class enrollment must not exceed the Class's capacity when a registration is accepted.	Registration transaction and capacity validation
25	Enrollment reports count registered Players by Class, while attendance reports calculate participation using recorded session attendance.	SQL JOIN, COUNT, GROUP BY and aggregate queries
26	Only authorized users may create, update, or access academy records according to their assigned roles.	Authentication and role-based authorization in application

4. ERD and Tables

The conceptual model of the Football Academy Management System (FAMS) is represented by an Enhanced Entity-Relationship (EER) diagram using Chen notation with cardinality indicators. The model contains **14 core entities**, including associative entities for registrations, attendance, and guardian relationships.
POSITION is specialized into GOALKEEPER and OUTFIELD_PLAYER. The intended specialization is disjoint and total, meaning each position belongs to exactly one subtype. This structure supports the classification of player positions while maintaining a shared POSITION entity.

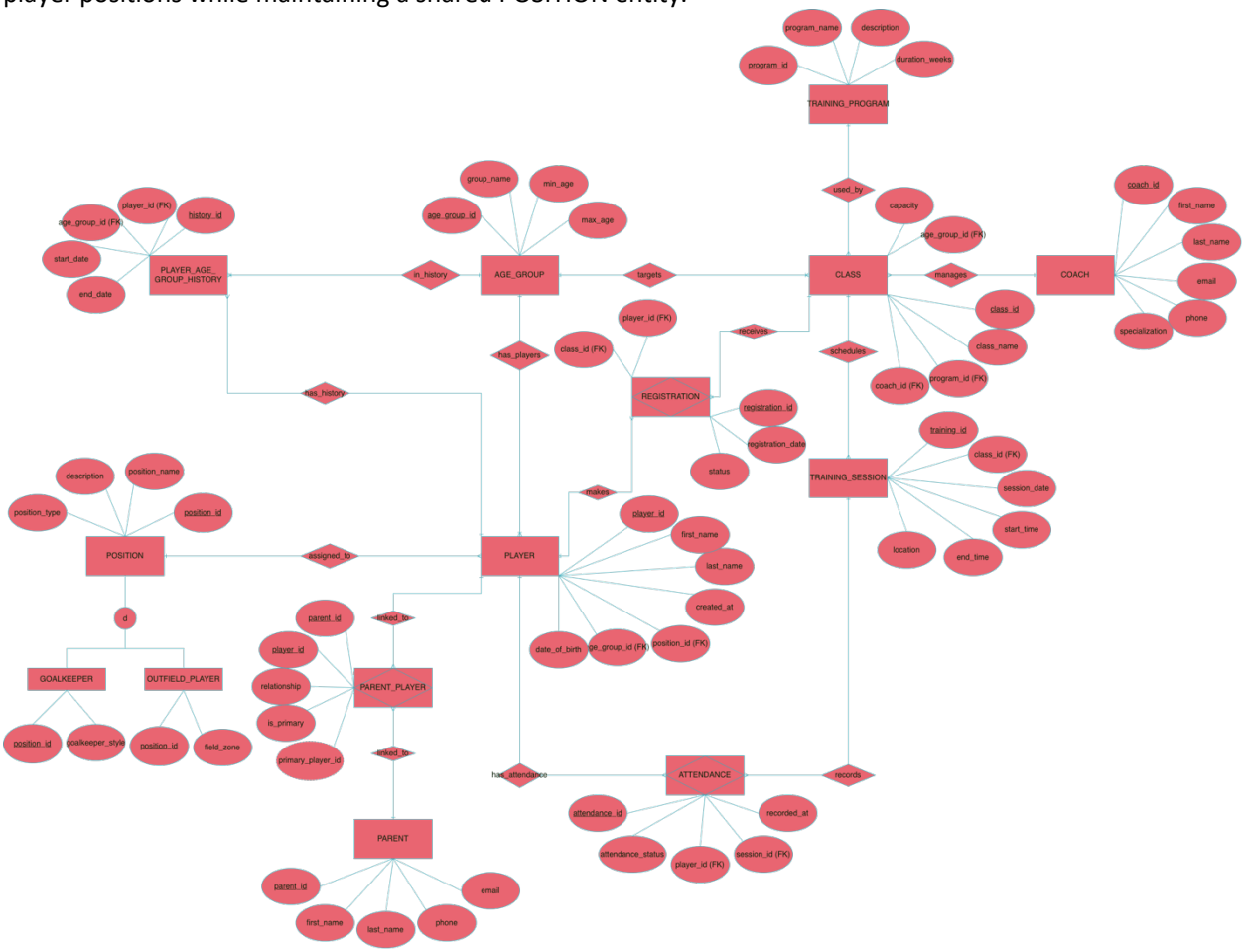


Figure 1. EER diagram of Football Academy Management System (FAMS)

Entities and relationships

- AGE_GROUP has many PLAYERs and CLASSes. Each player and class belongs to one age group.
- POSITION has many PLAYERs and specializes into GOALKEEPER and OUTFIELD_PLAYER.
- PARENT and PLAYER have a many-to-many relationship through PARENT_PLAYER, which records the guardian relationship and primary guardian status.
- COACH manages many CLASSes, while each class is assigned to one coach.
- TRAINING_PROGRAM is associated with many CLASSes, while each class follows one training program.
- CLASS schedules many TRAINING_SESSIONs. Each training session belongs to one class.
- PLAYER enrolls in CLASS through REGISTRATION, which stores the registration date and status.

- PLAYER attendance is recorded for TRAINING_SESSION through ATTENDANCE, which stores attendance status and recording time.
- PLAYER_AGE_GROUP_HISTORY records a player's age group history, including start and end dates.

Physical tables (MySQL)

Table	Primary key	Foreign keys	Purpose
age_group	age_group_id	-	Defines age groups and their minimum and maximum ages
coach	coach_id	-	Stores coach profiles, contact information, and specializations.
position	position_id	-	Defines football positions and their types.
goalkeeper	position_id	position_id	Stores goalkeeper-specific information.
outfield_player	position_id	position_id	Stores outfield position-specific information
training_program	program_id	-	Stores training programs and their duration
parent	parent_id	-	Stores parent and guardian contact information
player	player_id	age_group_id; position_id	Stores player profiles and assigned age groups and positions
parent_player	(parent_id, player_id)	parent_id; player_id	Links guardians to players and identifies primary guardians
class	class_id	coach_id; program_id; age_group_id	Stores academy classes, assigned coaches, programs, and capacity
training_session	session_id	class_id	Stores scheduled training sessions, times, and locations
registration	registration_id	player_id; class_id	Records player enrollment and registration status
attendance	attendance_id	player_id; session_id	Records player attendance for training sessions
player_age_group_history	history_id	player_id; age_group_id	Tracks changes in players' age group assignments
app_user	*See authentication schema*	*See authentication schema*	Supporting: stores user accounts and access roles
age_group_recommendation	*See recommendation schema*	*See recommendation schema*	Supporting: stores or supports player age group recommendations

The database uses primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and triggers to maintain data integrity. Additional business rules, such as class capacity limits and role-based access control, are enforced through the Node.js/Express application.

The initial demonstration dataset consists of **120 players, 8 coaches, 12 classes, 4 age groups, and 36 training sessions in a sample week**. These values represent the initial dataset rather than fixed system limits.

5. Normalization Steps and Functional Dependencies

We normalized the football academy's business forms step by step to reduce data redundancy and prevent insertion, update, and deletion anomalies. Primary keys are underlined in the final document; braces { } indicate repeating groups.

Step 0. Unnormalized form (UNF): three business forms

PLAYER_ENROLLMENT_FORM(player_id, first_name, last_name, date_of_birth, age_group_id, group_name, min_age, max_age, position_id, position_name, position_type, created_at, {parent_id, parent_first_name, parent_last_name, phone, email, relationship, is_primary}, {registration_id, class_id, class_name, coach_id, coach_first_name, coach_last_name, program_id, program_name, duration_weeks, capacity, registration_date, registration_status})

TRAINING_ATTENDANCE_FORM(class_id, class_name, coach_id, coach_name, program_id, program_name, {session_id, session_date, start_time, end_time, location, {attendance_id, player_id, player_name, attendance_status, recorded_at}})

PLAYER_DEVELOPMENT_FORM(player_id, player_name, position_id, position_name, position_type, goalkeeper_style, field_zone, {history_id, age_group_id, group_name, start_date, end_date})

Step 1. First normal form (1NF): each repeating group becomes a relation with its parent's key

Each repeating group is separated into its own relation. Every attribute contains a single value, and each row is uniquely identifiable.

PLAYER_1 (player_id, first_name, last_name, date_of_birth, age_group_id, group_name, min_age, max_age, position_id, position_name, position_type, created_at)

PARENT_PLAYER_1 (parent_id, player_id, parent_first_name, parent_last_name, phone, email, relationship, is_primary)

REGISTRATION_1 (registration_id, player_id, class_id, class_name, coach_id, coach_first_name, coach_last_name, program_id, program_name, duration_weeks, capacity, registration_date, registration_status)

TRAINING_SESSION_1 (session_id, class_id, class_name, coach_id, coach_name, program_id, program_name, session_date, start_time, end_time, location)

ATTENDANCE_1 (attendance_id, player_id, player_name, session_id, attendance_status, recorded_at)

PLAYER_AGE_GROUP_HISTORY_1 (history_id, player_id, player_name, age_group_id, group_name, start_date, end_date)

Step 2. Second normal form (2NF): remove partial dependencies (composite keys only)

Second Normal Form requires every non-key attribute to depend on the entire candidate key.

- PARENT_PLAYER_1: parent_id → parent_first_name, parent_last_name, phone, email. Guardian details depend only on parent_id, not on the full composite key (parent_id, player_id). These attributes are moved to PARENT.

  PARENT (parent_id, first_name, last_name, phone, email)
  PARENT_PLAYER (parent_id, player_id, relationship, is_primary)
- REGISTRATION_1: The combination (player_id, class_id) is a candidate key. Player and class information are maintained in their respective relations instead of being repeated in registration records.

  REGISTRATION (registration_id, player_id, class_id, registration_date, status)

- ATTENDANCE_1: The combination (player_id, session_id) is a candidate key. Player details and training session details are stored separately.

  ATTENDANCE (attendance_id, player_id, session_id, status, recorded_at)

  The remaining relations use single-attribute primary keys and have no partial dependencies on their primary keys.

Step 3. Third normal form (3NF): remove transitive dependencies

The following dependencies identify attributes that belong to separate entities.

- age_group_id -> group_name, min_age, max_age: Extract **AGE_GROUP**.
- position_id -> position_name, position_type: Extract **POSITION**.
- coach_id -> first_name, last_name, email, phone, specialization: Extract **COACH**.
- program_id -> program_name, description, duration_weeks: Extract **TRAINING_PROGRAM**.
- class_id -> class_name, coach_id, program_id, age_group_id, capacity: Extract **CLASS**.
- session_id -> class_id, session_date, start_time, end_time, location: Extract **TRAINING_SESSION**.
- position_id -> goalkeeper_style for goalkeeper positions: Extract **GOALKEEPER**.
- position_id -> field_zone for outfield positions: Extract **OUTFIELD_PLAYER**.

After removing these dependencies, the final 14 core relations are:

AGE_GROUP (age_group_id, group_name, min_age, max_age)

COACH (coach_id, first_name, last_name, email, phone, specialization)

POSITION (position_id, position_name, position_type)

GOALKEEPER (position_id, goalkeeper_style)

OUTFIELD_PLAYER (position_id, field_zone)

TRAINING_PROGRAM (program_id, program_name, description, duration_weeks)

PARENT (parent_id, first_name, last_name, phone, email)

PLAYER (player_id, first_name, last_name, date_of_birth, age_group_id, position_id, created_at)

PARENT_PLAYER (parent_id, player_id, relationship, is_primary)

CLASS (class_id, class_name, coach_id, program_id, age_group_id, capacity)

TRAINING_SESSION (session_id, class_id, session_date, start_time, end_time, location)

REGISTRATION (registration_id, player_id, class_id, registration_date, status)

ATTENDANCE (attendance_id, player_id, session_id, status, recorded_at)

PLAYER_AGE_GROUP_HISTORY (history_id, player_id, age_group_id, start_date, end_date)

The generated attribute primary_player_id in parent_player is omitted from this conceptual normalization listing because it is an implementation mechanism for enforcing the primary guardian constraint.

Step 4. Boyce-Codd normal form (BCNF): functional dependencies of the final relations

Relation	Functional dependencies
AGE_GROUP	age_group_id → group_name, min_age, max_age; group_name → age_group_id
COACH	coach_id → first_name, last_name, email, phone, specialization; email → coach_id

Relation	Functional dependencies
POSITION	position_id → position_name, position_type; position_name → position_id
GOALKEEPER	position_id → goalkeeper_style
OUTFIELD_PLAYER	position_id → field_zone
TRAINING_PROGRAM	program_id → program_name, description, duration_weeks; program_name → program_id
PARENT	parent_id → first_name, last_name, phone, email
PLAYER	player_id → first_name, last_name, date_of_birth, age_group_id, position_id, created_at
PARENT_PLAYER	(parent_id, player_id) → relationship, is_primary
CLASS	class_id → class_name, coach_id, program_id, age_group_id, capacity
TRAINING_SESSION	session_id → class_id, session_date, start_time, end_time, location
REGISTRATION	registration_id → player_id, class_id, registration_date, status; (player_id, class_id) → registration_id, registration_date, status
ATTENDANCE	attendance_id → player_id, session_id, status, recorded_at; (player_id, session_id) → attendance_id, status, recorded_at
PLAYER_AGE_GROUP_HISTORY	history_id → player_id, age_group_id, start_date, end_date

Under the stated functional dependencies, each non-trivial determinant is a candidate key or superkey of its relation. Therefore, the 14 core relations are designed to satisfy BCNF.

The database also uses foreign keys, unique constraints, checks, triggers, and application-level validation to enforce additional business rules. These mechanisms complement normalization but are not themselves normalization steps.

6. Mapping (EER to Relational Schema)

We mapped the Enhanced Entity-Relationship (EER) diagram of the Football Academy Management System (FAMS) to MySQL relational tables using the standard nine-step mapping algorithm described by Elmasri and Navathe.

This section explains how entities, relationships, primary keys, foreign keys, and specialization structures were converted into the relational schema.

The final design contains 14 core relations and two supporting relations used for authentication and age group recommendations.

Conventions: Table and column names use lowercase snake_case. Primary keys are underlined in the final document, and foreign keys are identified explicitly. The database uses MySQL 8+ with primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and business-rule triggers.

6.1 Overview of the mapping

EER construct	Kind	Mapping step	Result in the relational schema
AGE_GROUP, COACH, POSITION, TRAINING_PROGRAM, PARENT, PLAYER, CLASS	Regular entities	Step 1	One table for each entity
None	Weak entities	Step 2	No weak-entity mapping required
None in the 14 core entities	Binary 1:1	Step 3	No separate 1:1 relationship table
AGE_GROUP–PLAYER, AGE_GROUP–CLASS, POSITION–PLAYER, COACH–CLASS, TRAINING_PROGRAM–CLASS, CLASS–TRAINING_SESSION	Binary 1:N relationships	Step 4	Foreign key placed on the N side
PARENT–PLAYER, PLAYER–CLASS, PLAYER–TRAINING_SESSION	Binary M:N relationships	Step 5	PARENT_PLAYER, REGISTRATION, ATTENDANCE
None	Multivalued attributes	Step 6	No additional table
None	N-ary relationships	Step 7	No additional table
POSITION → GOALKEEPER / OUTFIELD_PLAYER	Specialization	Step 8	Option 8A: superclass and subtype tables
None	Union types	Step 9	Not used
Authentication and recommendations	Supporting features	Additional	APP_USER, AGE_GROUP_RECOMMENDATION

6.2 Step 1: Regular entity types

Rule. For each regular entity, create a relation containing its simple attributes, select a primary key, and preserve applicable uniqueness constraints.

Applied. The main independent entities of FAMS are AGE_GROUP, COACH, POSITION, TRAINING_PROGRAM, PARENT, PLAYER, and CLASS.

The database uses numeric identifiers to distinguish records even when names or other descriptive attributes change.

AGE_GROUP (age_group_id, group_name, min_age, max_age)

Column	Type	Key	Constraints and notes
age_group_id	INT	PK	AUTO_INCREMENT
group_name	VARCHAR(60)	UNIQUE	NOT NULL
min_age	INT	-	NOT NULL; >= 0
max_age	INT	-	NOT NULL; > min_age

COACH (coach_id, first_name, last_name, email, phone, specialization)

Column	Type	Key	Constraints and notes
coach_id	INT	PK	AUTO_INCREMENT
first_name	VARCHAR(80)	-	NOT NULL
last_name	VARCHAR(80)	-	NOT NULL
email	VARCHAR(150)	UNIQUE	Nullable
phone	VARCHAR(30)	-	Nullable
specialization	VARCHAR(120)	-	Nullable

POSITION (position_id, position_name, position_type)

Column	Type	Key	Constraints and notes
position_id	INT	PK	AUTO_INCREMENT
position_name	VARCHAR(80)	UNIQUE	NOT NULL
position_type	ENUM('Goalkeeper','Outfield')	-	NOT NULL; subtype discriminator

TRAINING_PROGRAM (program_id, program_name, description, duration_weeks)

Column	Type	Key	Constraints and notes
program_id	INT	PK	AUTO_INCREMENT
program_name	VARCHAR(120)	UNIQUE	NOT NULL
description	TEXT	-	Nullable
duration_weeks	INT	-	NOT NULL DEFAULT 1; > 0

PARENT (parent_id, first_name, last_name, phone, email)

Column	Type	Key	Constraints and notes
parent_id	INT	PK	AUTO_INCREMENT
first_name	VARCHAR(80)	-	NOT NULL
last_name	VARCHAR(80)	-	NOT NULL
phone	VARCHAR(30)	-	NOT NULL
email	VARCHAR(150)	-	Nullable

PLAYER (player_id, first_name, last_name, date_of_birth, age_group_id, position_id, created_at)

Column	Type	Key	Constraints and notes
player_id	INT	PK	AUTO_INCREMENT
first_name	VARCHAR(80)	-	NOT NULL
last_name	VARCHAR(80)	-	NOT NULL
date_of_birth	DATE	-	NOT NULL; >= 1900-01-01
age_group_id	INT	FK	NOT NULL → AGE_GROUP
position_id	INT	FK	NOT NULL → POSITION
created_at	TIMESTAMP	-	DEFAULT CURRENT_TIMESTAMP

CLASS (class_id, class_name, coach_id, program_id, age_group_id, capacity)

Column	Type	Key	Constraints and notes
class_id	INT	PK	AUTO_INCREMENT
class_name	VARCHAR(120)	-	NOT NULL
coach_id	INT	FK	NOT NULL → COACH
program_id	INT	FK	NOT NULL → TRAINING_PROGRAM
age_group_id	INT	FK	NOT NULL → AGE_GROUP
capacity	INT	-	NOT NULL DEFAULT 30; > 0

6.3 Step 2: Weak entity types

Rule. A weak entity is mapped to a relation containing its own attributes and the primary key of its identifying owner. Its primary key normally combines the owner's key with a partial key.

Applied. Although TRAINING_SESSION depends on CLASS through a foreign key, it has its own independent primary key, session_id. Similarly, PLAYER_AGE_GROUP_HISTORY uses history_id.

Therefore, these relations are not mapped as weak entities.

6.4 Step 3: Binary 1:1 relationship types

Rule. A binary 1:1 relationship can be mapped by placing the primary key of one entity as a foreign key in the other, preferably on the side with total participation.

Applied. The 14 core FAMS entities do not contain a standalone binary 1:1 relationship requiring this mapping.

The primary guardian restriction is handled through PARENT_PLAYER. A player may have several guardians, but no more than one can be marked as primary.

This is a constraint on an M relationship rather than a separate 1:1 relationship.

6.5 Step 4: Binary 1:N Relationship Types

Rule. For each binary 1:N relationship, place the primary key of the entity on the 1 side as a foreign key in the entity on the N side.

Relationship (1 : N)	Foreign key placed in N side	Explanation
AGE_GROUP → PLAYER	player.age_group_id	Each player belongs to one age group
POSITION → PLAYER	player.position_id	Each player has one assigned position
AGE_GROUP → CLASS	class.age_group_id	Each class targets one age group
COACH → CLASS	class.coach_id	Each class is assigned to one coach
TRAINING_PROGRAM → CLASS	class.program_id	Each class follows one training program
CLASS → TRAINING_SESSION	training_session.class_id	Each session belongs to one class
PLAYER → AGE GROUP HISTORY	player_age_group_history.player_id	A player can have multiple history records
AGE_GROUP → AGE GROUP HISTORY	player_age_group_history.age_group_id	Each history record refers to one age group

TRAINING_SESSION (session_id, class_id, session_date, start_time, end_time, location)

Column	Type	Key	Constraints and notes
session_id	INT	PK	AUTO_INCREMENT
class_id	INT	FK	NOT NULL → CLASS; ON DELETE CASCADE
session_date	DATE	-	NOT NULL
start_time	TIME	-	NOT NULL
end_time	TIME	-	NOT NULL; > start_time
location	VARCHAR(150)	-	Nullable

PLAYER_AGE_GROUP_HISTORY (history_id, player_id, age_group_id, start_date, end_date)

Column	Type	Key	Constraints and notes
history_id	INT	PK	Unique history record
player_id	INT	FK	References PLAYER
age_group_id	INT	FK	References AGE_GROUP
start_date	DATE	-	Beginning of age group assignment
end_date	DATE	-	Ending date, when applicable

6.6 Step 5: Binary M:N relationship types and associative entities

Rule. For each binary M:N relationship, create an associative relation containing foreign keys referencing the participating entities and any attributes describing the relationship.

Applied.FAMS has three major M:N relationships: PARENT–PLAYER, PLAYER–CLASS, and PLAYER–TRAINING_SESSION. These are mapped through PARENT_PLAYER, REGISTRATION, and ATTENDANCE, respectively.

PARENT and PLAYER: PARENT_PLAYER

A guardian may be associated with multiple players, and a player may have multiple guardians.

PARENT_PLAYER (parent_id, player_id, relationship, is_primary, primary_player_id)

Column	Type	Key	Constraints and notes
parent_id	INT	PK,FK	References PARENT
player_id	INT	PK,Fk	References PLAYER
relationship	VARCHAR(50)	-	Relationship to player
is_primary	BOOLEAN (TINYINT(1))	-	Primary guardian indicator
primary_player_id	INT	UNIQUE	Generated column used to enforce one primary guardian per player

The composite primary key (parent_id, player_id) prevents duplicate guardian-player relationships.The generated column supports the restriction that a player cannot have more than one primary guardian

PLAYER and CLASS: REGISTRATION

A player may register for multiple classes, and each class may contain multiple players.

REGISTRATION (registration_id, player_id, class_id, registration_date, status)

Column	Type	Key	Constraints and notes
registration_id	INT	PK	Unique registration identifier
player_id	INT	FK	References PLAYER
class_id	INT	FK	References CLASS
registration_date	DATE	-	Registration date
status	ENUM('Active','Pending','Cancelled','Completed')	-	Active, Pending, Cancelled, Completed
(player_id, class_id)	-	UNIQUE	One registration per player-class pair

A surrogate primary key allows the system to identify registration records directly, while the UNIQUE constraint preserves the business rule that a player cannot register for the same class twice.Additional validation checks the player's age group and class capacity.

PLAYER and TRAINING_SESSION: ATTENDANCE

A player may attend many training sessions, and each session may have many players.

ATTENDANCE (attendance_id, player_id, session_id, status, recorded_at)

Column	Type	Key	Constraints and notes
attendance_id	INT	PK	Unique attendance identifier
player_id	INT	FK	References PLAYER
session_id	INT	FK	References TRAINING_SESSION
status	ENUM('Present','Absent','Late', 'Excused')	-	Present, Absent, Late, Excused
recorded_at	TIMESTAMP	-	Time the attendance was recorded
(player_id, session_id)	-	UNIQUE	Prevents duplicate attendance records

Attendance validation must also confirm that the player is registered for the class associated with the training session.

6.7 Step 6: Multivalued attributes

Rule A multivalued attribute is mapped to a separate relation containing the owning entity's key and the attribute value.

Applied. The FAMS EER does not contain a standalone multivalued attribute requiring this transformation. For example, a player can have multiple guardians, but guardians are modeled as independent PARENT entities connected through PARENT_PLAYER rather than as a repeating attribute.Therefore, no additional relation is created in this step.

6.8 Step 7: N-ary relationship types

Rule. An N-ary relationship involving more than two participating entity types is mapped to a relation containing foreign keys to the participating entities.

Applied. The FAMS EER uses binary relationships and associative entities.For example, ATTENDANCE connects PLAYER and TRAINING_SESSION. The associated CLASS can be identified through TRAINING_SESSION, so CLASS does not need to be repeated as an additional foreign key in ATTENDANCE. Therefore, no separate N-ary relationship mapping is required.

6.9 Step 8: Specialization and generalization

Rule. An EER specialization can be mapped using several approaches:

- **8A:** One relation for the superclass and one relation for each subclass.
- **8B:** Relations for subclasses only, repeating superclass attributes.
- **8C:** One superclass relation containing a type discriminator.
- **8D:** One superclass relation containing multiple subtype indicators.

FAMS Specialization

The FAMS EER includes the following hierarchy:

POSITION → GOALKEEPER / OUTFIELD_PLAYER

The intended specialization is:

- Disjoint: A position belongs to only one subtype.
- Total: Every position should belong to either GOALKEEPER or OUTFIELD_PLAYER.

Comparison of Mapping Options

Option	Tables it would create	Pros	Cons
8A (chosen)	POSITION, GOALKEEPER, OUTFIELD_PLAYER	Follows the EER structure exactly; stores subtype-specific attributes separately; avoids unnecessary NULL values	Requires JOIN operations when retrieving subtype details; additional validation is needed to enforce disjoint and total specialization
8B	GOALKEEPER, OUTFIELD_PLAYER	Subtype details can be retrieved without joining a superclass table	Repeats common POSITION attributes; makes it harder to maintain a single position identifier across both tables
8C	POSITION with position_type, goalkeeper_style, field_zone	Uses only one table; simplifies queries; a discriminator identifies each subtype	Subtype-specific columns may contain NULL values; additional constraints are needed
8D	POSITION with is_goalkeeper, is_outfield, goalkeeper_style, field_zone	Can represent overlapping subclasses	Unnecessary for disjoint specialization; conflicting Boolean values must be prevented

Decision.

- We selected Option 8A for the POSITION specialization.
- The superclass POSITION stores common attributes: position_id, position_name, and position_type.
- GOALKEEPER stores the goalkeeper-specific attribute goalkeeper_style.
- OUTFIELD_PLAYER stores the outfield-specific attribute field_zone.
- Each subclass uses position_id as both its primary key and foreign key referencing POSITION.
- The position_type attribute identifies the intended subtype, while additional integrity validation is required to guarantee that each position belongs to exactly one matching subtype.

Specialization Mapping

Hierarchy	Superclass	Subclass	Primary / Foreign Key	Description
POSITION specialization	POSITION	GOALKEEPER	goalkeeper.position_id → position.position_id	goalkeeper_style
POSITION specialization	POSITION	OUTFIELD_PLAYER	outfield_player.position_id → position.position_id	field_zone

Subtype Behavior

Subtype	position_type	Specific Attribute	Description
GOALKEEPER	Goalkeeper	goalkeeper_style	Represents goalkeeper-specific playing characteristics
OUTFIELD_PLAYER	Outfield	field_zone	Represents the area of the field associated with an outfield position

Resulting Relations

POSITION (position_id, position_name, position_type)
GOALKEEPER (position_id, goalkeeper_style)
OUTFIELD_PLAYER (position_id, field_zone)

The use of Option 8A preserves the specialization structure of the conceptual EER model and allows each subtype to maintain its own attributes without duplicating common position information.

6.10 Step 9: Union types (categories)

Rule. A union type represents a subclass whose members can originate from multiple different superclasses. It can be mapped using a separate relation with an appropriate identifier and references to the participating entity types.

Applied. The FAMS EER does not contain union types.GOALKEEPER and OUTFIELD_PLAYER are subclasses of the same superclass, POSITION. Therefore, they represent specialization rather than a union category.

No additional relation was created in this step.

6.11 Supporting tables outside the EER

Two supporting relations extend the functionality of the Football Academy Management System beyond the 14 core relations.These tables support user authentication, role-based access control, and player age group recommendations.

APP_USER

The APP_USER table manages system accounts and associates users with their relevant coach or parent records

APP_USER (user_id, username, password_hash, role, coach_id, parent_id, created_at)

Column	Type	Key	Constraints and notes
user_id	INT	PK	Unique user identifier; AUTO_INCREMENT
username	VARCHAR	UNIQUE	Unique login username
password_hash	VARCHAR	-	Stores the hashed password
role	VARCHAR / ENUM	-	identifies the user's application role
coach_id	INT	FK, UNIQUE	Nullable; references COACH
parent_id	INT	FK, UNIQUE	Nullable; references PARENT
created_at	TIMESTAMP / DATETIME	-	Account creation timestamp

The APP_USER table supports authentication and authorization. The nullable foreign keys allow an account to be associated with the appropriate type of academy user.

AGE_GROUP_RECOMMENDATION

The AGE_GROUP_RECOMMENDATION table records recommendations made by coaches concerning a player's age group assignment.

AGE_GROUP_RECOMMENDATION (recommendation_id, player_id, coach_id, suggested_age_group_id, reason, status, created_at, reviewed_at, reviewed_by)

Column	Type	Key	Constraints and notes
recommendation_id	INT	PK	Unique recommendation identifier; AUTO_INCREMENT
player_id	INT	FK	References PLAYER
coach_id	INT	FK	References COACH

Column	Type	Key	Constraints and notes
suggested_age_group_id	INT	FK	References AGE_GROUP
reason	TEXT	-	Explanation for the recommendation
status	VARCHAR / ENUM	-	Recommendation review status
created_at	TIMESTAMP / DATETIME	-	Recommendation creation timestamp
reviewed_at	TIMESTAMP / DATETIME	-	Nullable; time of review
reviewed_by	INT	FK	References the reviewing account or authorized record, as defined in the SQL schema

These two relations support additional application requirements without changing the fundamental relationships among the 14 core entities.

Note: Exact VARCHAR lengths, ENUM definitions, timestamp types, and the target of reviewed_by must be checked against the supporting SQL migrations before final submission.

6.12 Final relational schema

The EER mapping produces 14 core relations and 2 supporting relations, for a total of 16 application relations.

Core Relations

1. **AGE_GROUP** (age_group_id, group_name, min_age, max_age)
2. **COACH** (coach_id, first_name, last_name, email, phone, specialization)
3. **POSITION** (position_id, position_name, position_type)
4. **GOALKEEPER** (position_id, goalkeeper_style)
5. **OUTFIELD_PLAYER** (position_id, field_zone)
6. **TRAINING_PROGRAM** (program_id, program_name, description, duration_weeks)
7. **PARENT** (parent_id, first_name, last_name, phone, email)
8. **PLAYER** (player_id, first_name, last_name, date_of_birth, age_group_id, position_id, created_at)
9. **PARENT_PLAYER** (parent_id, player_id, relationship, is_primary, primary_player_id)
10. **CLASS** (class_id, class_name, coach_id, program_id, age_group_id, capacity)
11. **TRAINING_SESSION** (session_id, class_id, session_date, start_time, end_time, location)
12. **REGISTRATION** (registration_id, player_id, class_id, registration_date, status)
13. **ATTENDANCE** (attendance_id, player_id, session_id, status, recorded_at)
14. **PLAYER_AGE_GROUP_HISTORY** (history_id, player_id, age_group_id, start_date, end_date)

Supporting Relations

1. **APP_USER** (user_id, username, password_hash, role, coach_id, parent_id, created_at)
2. **AGE_GROUP_RECOMMENDATION** (recommendation_id, player_id, coach_id, suggested_age_group_id, reason, status, created_at, reviewed_at, reviewed_by)

Indexes Added After Mapping

MySQL automatically creates indexes for primary keys and unique constraints. Foreign key columns must also have suitable indexes; MySQL can create a required index when one does not already exist.

The FAMS database uses or may benefit from indexes supporting common relationships and operations, including:

- Player searches by age group and position.
- Class searches by coach, program, and age group.
- Training session retrieval by class and date.
- Registration lookup by player and class.
- Attendance lookup by player and training session.
- Guardian-player relationship lookup.

The exact index names and definitions should be taken from the implemented SQL schema.

Rule	Mechanism	Why not a key or CHECK
A player has at most one primary guardian	Generated column with UNIQUE constraint in PARENT_PLAYER	The restriction applies only to guardian relationships marked as primary
A player must belong to a valid age group	Foreign key and age validation	A foreign key cannot compare the player's age with the age group's boundaries
Class enrollment must not exceed capacity	Transaction-safe validation in the application or database	Requires counting active registrations for a class
A player cannot register for the same class twice	UNIQUE (player_id, class_id)	Enforced directly by a composite unique constraint
Attendance must correspond to a valid registration	Database trigger or application validation	Requires checking registration and the class associated with a session
A player cannot have duplicate attendance for one session	UNIQUE (player_id, session_id)	Enforced directly by a composite unique constraint
Every position must belong to exactly one correct subtype	Subtype consistency validation	Separate subclass foreign keys do not automatically guarantee total and disjoint specialization
Only authorized users may perform restricted operations	Application-level role-based access control	Authorization depends on the authenticated user's role

Final Mapping Summary

The final FAMS relational design applies the nine-step EER mapping algorithm to transform the conceptual database model into MySQL relations.

The design uses:

- Separate tables for regular entities.
- Foreign keys for 1:N relationships.
- Associative tables for M:N relationships.
- Option 8A for the POSITION specialization.
- Additional supporting tables for authentication and age group recommendations.
- Database constraints and application validation for complex business rules.

This structure reduces data redundancy, preserves referential integrity, and supports the operational requirements of the Football Academy Management System.

7. DDL and DML

The Football Academy Management System (FAMS) database uses MySQL 8+ to manage player information, guardians, coaches, training programs, classes, training sessions, registrations, attendance, and player age group history.

The main database schema is defined in database/01_schema.sql. Additional SQL scripts support business-rule validation, authentication, and age group recommendations.

This section presents representative Data Definition Language (DDL) statements and Data Manipulation Language (DML) queries used to demonstrate the database structure and core academy operations.

DDL: Training Classes and Registration

```
CREATE TABLE class (
	class_id INT AUTO_INCREMENT PRIMARY KEY,
	class_name VARCHAR(120) NOT NULL,
	coach_id INT NOT NULL,
	program_id INT NOT NULL,
	age_group_id INT NOT NULL,
	capacity INT NOT NULL DEFAULT 30,

	CONSTRAINT fk_class_coach
		FOREIGN KEY (coach_id)
		REFERENCES coach(coach_id),

	CONSTRAINT fk_class_program
		FOREIGN KEY (program_id)
		REFERENCES training_program(program_id),

	CONSTRAINT fk_class_age_group
		FOREIGN KEY (age_group_id)
		REFERENCES age_group(age_group_id),

	CONSTRAINT chk_class_capacity
		CHECK (capacity > 0)
);
```

The CLASS table stores each training class and references its assigned coach, training program, and age group. The capacity attribute specifies the maximum number of players allowed in each class and must be greater than zero.

```
CREATE TABLE registration (
    registration_id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT NOT NULL,
    class_id INT NOT NULL,
    registration_date DATE NOT NULL,
    status ENUM(
        'Active',
        'Pending',
        'Cancelled',
        'Completed'
    ) NOT NULL DEFAULT 'Active',

    CONSTRAINT uq_player_class
        UNIQUE (player_id, class_id),

    CONSTRAINT fk_registration_player
        FOREIGN KEY (player_id)
        REFERENCES player(player_id),

    CONSTRAINT fk_registration_class
        FOREIGN KEY (class_id)
        REFERENCES class(class_id)
);
```

The REGISTRATION table represents the many-to-many relationship between PLAYER and CLASS. The unique constraint prevents a player from registering for the same class more than once.

DDL: Training Sessions and Attendance

```
CREATE TABLE training_session (
    session_id INT AUTO_INCREMENT PRIMARY KEY,
    class_id INT NOT NULL,
    session_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    location VARCHAR(150),

    CONSTRAINT fk_session_class
        FOREIGN KEY (class_id)
        REFERENCES class(class_id),

    CONSTRAINT chk_session_time
        CHECK (end_time > start_time)
);
```

The ATTENDANCE table records player participation in each training session.

```
CREATE TABLE attendance (
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT NOT NULL,
    session_id INT NOT NULL,
    status ENUM(
        'Present',
        'Absent',
        'Late',
        'Excused'
    ) NOT NULL,
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT uq_player_session
        UNIQUE (player_id, session_id),

    CONSTRAINT fk_attendance_player
        FOREIGN KEY (player_id)
        REFERENCES player(player_id),

    CONSTRAINT fk_attendance_session
        FOREIGN KEY (session_id)
        REFERENCES training_session(session_id)
);
```

The composite unique constraint (player_id, session_id) prevents duplicate attendance records for the same player and training session.

DML: Player Registration Transaction

The following transaction demonstrates how a player can be registered for a training class.

The application should verify that the player belongs to the correct age group and that the class has available capacity before completing the registration.

```
START TRANSACTION;

-- Lock the selected class during registration.
SELECT
    class_id,
    age_group_id,
    capacity
FROM class
WHERE class_id = ?
FOR UPDATE;
```

```
-- Check the number of active registrations.
SELECT COUNT(*) AS enrolled_players
FROM registration
WHERE class_id = ?
  AND status = 'Active';

-- Execute this INSERT only if age-group and capacity validation passes.
INSERT INTO registration (
    player_id,
    class_id,
    registration_date,
    status
)
VALUES (
    ?,
    ?,
    CURDATE(),
    'Active'
);

COMMIT;
```

The transaction illustrates the player registration workflow. Before executing the INSERT statement, the application must verify that the player's age group matches the selected class and that the number of active registrations is less than the class capacity. If either condition is not satisfied, the application must execute ROLLBACK instead of completing the registration. The SELECT ... FOR UPDATE statement locks the selected class row to help prevent concurrent registrations from exceeding the capacity.

DML: Recording Player Attendance

The following statement records attendance for a player participating in a training session.

```
INSERT INTO attendance (
    player_id,
    session_id,
    status,
    recorded_at
)
VALUES (
    ?,
    ?,
    'Present',
    CURRENT_TIMESTAMP
);
```

The system must verify that the player has a valid registration for the class associated with the session.

The unique constraint ensures that only one attendance record exists for each player-session combination.

DML: Management Reports

Report 1: Active Player Registrations by Class

```
SELECT
    c.class_id,
    c.class_name,
    ag.group_name AS age_group,
    CONCAT(
        co.first_name, ' ',
        co.last_name
    ) AS coach_name,
    c.capacity,
    COUNT(r.registration_id) AS enrolled_players
FROM class c
JOIN coach co
```

```
    ON c.coach_id = co.coach_id
JOIN age_group ag
    ON c.age_group_id = ag.age_group_id
LEFT JOIN registration r
    ON c.class_id = r.class_id
   AND r.status = 'Active'
GROUP BY
    c.class_id,
    c.class_name,
    ag.group_name,
    co.first_name,
    co.last_name,
    c.capacity
ORDER BY c.class_name;
```

This query shows the number of active registrations in each class, along with the assigned coach, age group, and maximum capacity.

Report 2: Player Attendance Summary

```
SELECT
    p.player_id,
    CONCAT(
        p.first_name, ' ',
        p.last_name
    ) AS player_name,
    COUNT(a.attendance_id) AS total_records,
    SUM(
        CASE
            WHEN a.status = 'Present'
            THEN 1 ELSE 0
        END
    ) AS present_count,
    SUM(
        CASE
            WHEN a.status = 'Absent'
            THEN 1 ELSE 0
        END
    ) AS absent_count,
    SUM(
        CASE
            WHEN a.status = 'Late'
            THEN 1 ELSE 0
        END
    ) AS late_count
FROM player p
LEFT JOIN attendance a
    ON p.player_id = a.player_id
GROUP BY
    p.player_id,
    p.first_name,
    p.last_name
ORDER BY player_name;
```

This report summarizes the attendance records of each player, including the number of present, absent, and late statuses.

Report 3: Upcoming Training Sessions

```
SELECT
    ts.session_id,
    c.class_name,
    ts.session_date,
    ts.start_time,
    ts.end_time,
    ts.location,
    CONCAT(
```

```
        co.first_name, ' ',
        co.last_name
    ) AS coach_name
FROM training_session ts
JOIN class c
    ON ts.class_id = c.class_id
JOIN coach co
    ON c.coach_id = co.coach_id
WHERE
    ts.session_date >= CURDATE()
ORDER BY
    ts.session_date,
    ts.start_time;
```

This query retrieves upcoming training sessions, including their scheduled dates, times, locations, class names, and assigned coaches.

Summary

The DDL statements define the academy's relational tables, primary keys, foreign keys, unique constraints, and validation rules.

The DML statements demonstrate the system's core operations, including player registration, attendance recording, enrollment reporting, attendance analysis, and training schedule retrieval.

Together, these SQL operations support data consistency and the daily management activities of the Football Academy Management System.

8. Database System Architecture

The Football Academy Management System (FAMS) follows a three-tier web application architecture consisting of the presentation layer, application layer, and database layer.

The system uses EJS templates to generate web pages, Node.js with Express.js to process HTTP requests and execute business logic, and MySQL to store and manage academy information.

The browser does not communicate directly with the database. Instead, requests are sent to the Express.js server, which validates input, processes academy operations, and executes SQL queries through the database connection.

Three-Tier Architecture

Presentation Layer — Web Browser

- Displays web pages generated using EJS templates.
- Provides forms for managing players, coaches, classes, registrations, and attendance.
- Sends HTTP requests to the application server.

Application Layer — Node.js and Express.js

- Receives and processes HTTP requests.
- Handles routing, authentication, and authorization.
- Validates submitted data.
- Executes business rules for academy operations.
- Communicates with MySQL through the database access layer.

Database Layer — MySQL

- Stores player, coach, guardian, training program, class, registration, and attendance data.
- Maintains relationships using primary keys and foreign keys.
- Enforces database constraints to protect data integrity.
- Processes SQL queries and returns results to the application server.

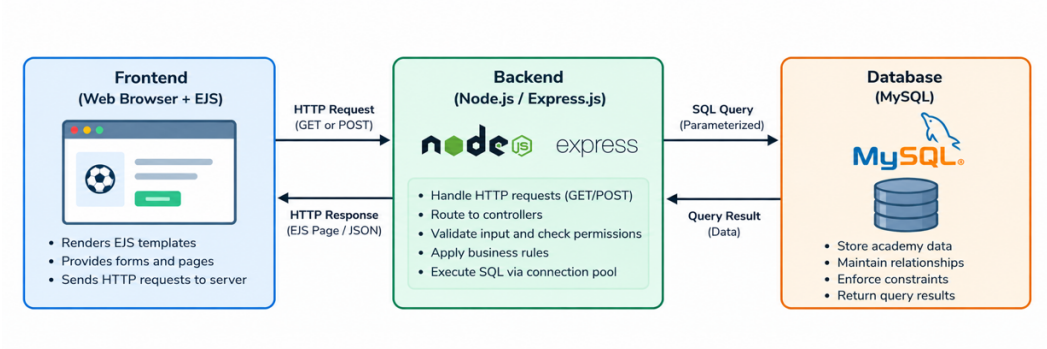


Figure 2. Three-tier architecture of FAMS: Web Browser (EJS), Node.js/Express.js Server, and MySQL Database.

How a request reaches the database

1. Browser. Users interact with the system through web pages rendered using EJS templates. Forms allow authorized users to submit information such as player details, class registrations, and attendance records. These requests are sent to the Express.js application server.

2. Route Protection. Protected application routes require authentication and appropriate user permissions. The application checks whether the user is authorized before allowing access to restricted functions such as managing players, classes, and attendance.

3. Express.js Routes. The Express.js server receives HTTP requests and directs them to the appropriate route handlers. GET requests retrieve information for display, while POST requests are used for operations such as creating registrations, updating player information, and recording attendance.

4. Business Logic. The application processes academy-specific rules before modifying the database. These rules include checking class capacity, validating age group eligibility, preventing duplicate registrations, and verifying attendance eligibility.

5. Data Access. The application communicates with MySQL through a database connection. SQL queries retrieve and modify records. Parameterized queries should be used to separate user input from SQL statements and reduce the risk of SQL injection.

6. MySQL Database. MySQL stores the relational tables defined in the database schema. Primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and any implemented triggers help maintain data integrity. Query results are returned to the Express.js server and displayed to users through EJS templates.

Example: Database Transaction for Class Registration

A registration operation may require several database checks before inserting a new record. A transaction helps ensure that the operation is completed consistently.

The following Node.js example illustrates the transaction structure using a MySQL connection pool.

```
async function transaction(work) {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        const result = await work(connection);

        await connection.commit();

        return result;
    } catch (error) {
        await connection.rollback();
```

```
        throw error;
    } finally {
        connection.release();
    }
}
```

The transaction helper obtains one database connection for the entire operation. If all statements succeed, the transaction is committed. If an error occurs, the transaction is rolled back to prevent incomplete database changes.

For example, a class registration operation can use this structure to lock the selected class record, check enrollment capacity, validate eligibility, and insert the registration before committing**.**

Concurrency Control

Concurrency control is important when multiple users attempt to register players for the same training class simultaneously.

For example, consider a training class with a maximum capacity of 20 players and 19 active registrations. If two users attempt to register another player at the same time, the system must prevent both requests from exceeding the class capacity.

MySQL InnoDB supports transactions and row-level locking. A registration workflow can use SELECT ... FOR UPDATE to lock the relevant class row while checking available capacity.

The second transaction must wait until the first transaction releases the lock. After the first transaction commits, the second transaction checks the updated enrollment count and rejects the registration if the class is already full.

This approach helps prevent over-enrollment when implemented consistently for all registration operations.

Architecture Summary

The three-tier architecture separates the user interface, application logic, and database management responsibilities.

The presentation layer provides the user interface through EJS templates. The application layer uses Node.js and Express.js to process requests and enforce business rules. The database layer uses MySQL to store relational data and maintain integrity constraints.

This separation improves maintainability, security, and scalability while supporting the daily operations of the Football Academy Management System.

9. Tech Stack

The Football Academy Management System (FAMS) is developed as a web-based application using Node.js, Express.js, EJS, and a MySQL relational database. The technologies are organized into the following layers.

Layer	Technology	Version	Purpose
Database	MySQL	8 (project requirement)	Tables, constraints, transactions, and academy report SQL
DB driver	mysql2	3.24.5	Connection pool and parameterized queries; no ORM
Web framework	Express.js	5.2.1	HTTP routes, middleware, controllers, and backend operations
Template engine	EJS	7.0.1	Server-rendered HTML pages and dynamic templates
Language	JavaScript	ECMAScript	Application logic, routes, services, and scripts
Runtime Environment	Node.js	18 or later (required)	Executes the Express.js server and application scripts
Authentication	express-session, bcryptjs	1.18.1 / 2.4.3	Login sessions, password hashing, and role-based access
Validation	express-validator	7.3.2	Validates and sanitizes user input
Styling	HTML5, CSS3, JavaScript	-	Responsive layouts, forms, tables, and interactive pages
Local database	MySQL local server	-	Local database development, schema creation, and testing
Scripts	dotenv, method-override, nodemon	18.0.6 / 3.0.0 / 3.1.14	Environment configuration, HTTP method handling, and automatic server restart
Unit and DB tests	Node.js test runner, mysql2	built in / 3.24.5	Authorization tests and database verification scripts
End-to-end tests	Manual browser testing	-	Verifies login, player management, registration, and attendance workflows
Code quality	Code review and testing scripts	-	Checks application behavior, SQL operations, and business rules
Design tools	MySQL Workbench	-	EER diagram, relational schema, and database design
Collaboration	Git and GitHub, VS Code	-	Version control, team collaboration, and source code development

Why this stack

- **MySQL for Relational Database Management:** MySQL supports primary keys, foreign keys, CHECK constraints, transactions, and SQL queries. These features help maintain data integrity and support the management of players, coaches, classes, registrations, and attendance records.
- **Node.js, Express.js, and EJS for Web Development:** Node.js runs the application server, Express.js handles HTTP requests and backend business logic, and EJS generates dynamic HTML pages. This combination provides a straightforward three-tier web architecture without requiring a separate frontend framework.
- **Plain SQL with mysql2 for Database Access:** Direct SQL queries allow the team to implement and examine joins, constraints, and transactions explicitly. This approach is particularly suitable for a database design project because it demonstrates SQL knowledge and provides clear control over database operations.

10. Milestones

The proposal planned six milestones by course week. The table shows the plan and what was actually delivered, with dates from our design documents and Git history.

Week	Planned milestone	Owner	Delivered	Date
Pitch	Proposal pitched and approved (Part B and 4-slide pitch)	Whole team	FAMS project proposal, system objectives, scope, user roles, 25 business rules, and preliminary EER model	1 Sep 2026
4	Complete EER diagram	Noppajit	EER diagram developed in Draw.io; 14 core entities, relationships, POSITION specialization, and revised business rules	Early Sep 2026
6	Relational schema and normalization (3NF)	Noppajit	Relational schema with primary and foreign keys; normalization from UNF to 3NF/BCNF; mapping of EER entities into relational tables	26 to 30 Sep 2026
7	Database built and seeded	Jakkaphat	MySQL database schema, DDL scripts, sample data, foreign key constraints, and database integrity rules	Sep 2026
8	SQL queries and management reports	Jakkaphat	SQL query scripts for player information, class enrollment, training schedules, and attendance reporting; target of at least 10 queries with results	5 to 6 Oct 2026
9	Demo and final report	Whole team	Node.js/Express.js web application with EJS interfaces, MySQL integration, role-based access, database documentation, final presentation slides, and system demonstration	Oct 2026

Quality checkpoints

- **Database:** Verify the 14 core relations, primary keys, foreign keys, UNIQUE constraints, sample data, and database business rules. Execute the SQL scripts and check that invalid records are rejected where appropriate.
- **Application:** Test the Express.js application, database connectivity, user authentication, role-based authorization, player management, class registration, training sessions, and attendance operations.
- **Reports:** Execute at least 10 SQL queries with results, as specified in the proposal. Verify player registration summaries, class enrollment information, and attendance reports.
- **Final Demonstration:** Present the EER diagram, normalization process, relational schema, SQL implementation, application interfaces, and management reports.

11. User Interface

We close with the actual screens of the Football Academy Management System (FAMS). Each interface demonstrates how the application uses MySQL data to support academy operations, including player management, class administration, registrations, attendance tracking, and SQL reporting.

Dashboard

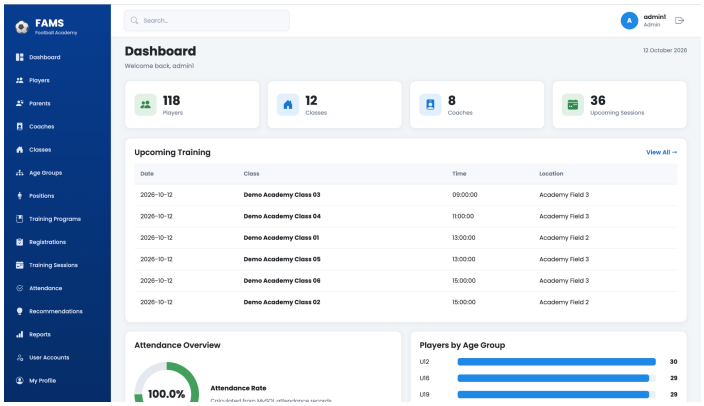


Figure 3. Academy overview and upcoming sessions.

Players

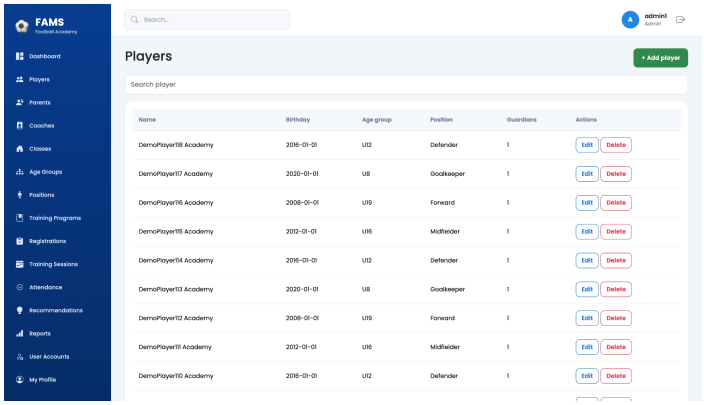


Figure 4. Search and manage player records.

Classes

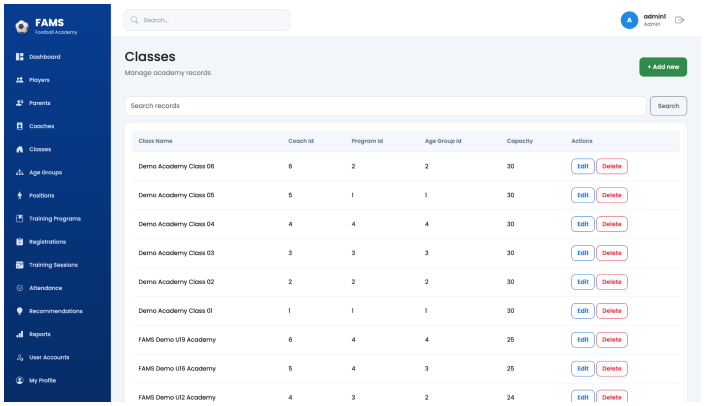


Figure 5. Manage classes, coaches, and capacities.

Registrations

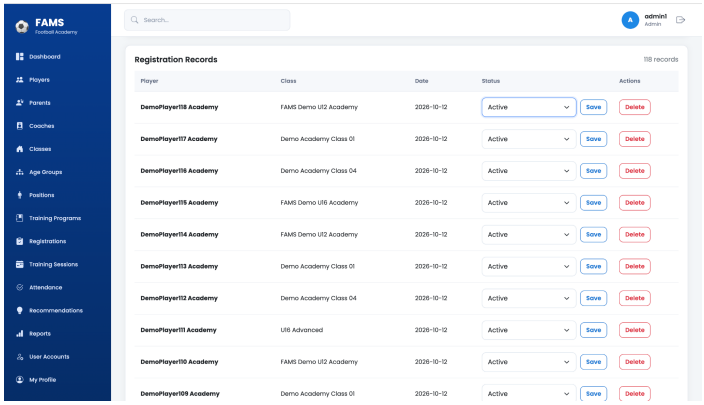


Figure 6. View and update player enrollments.

Attendance

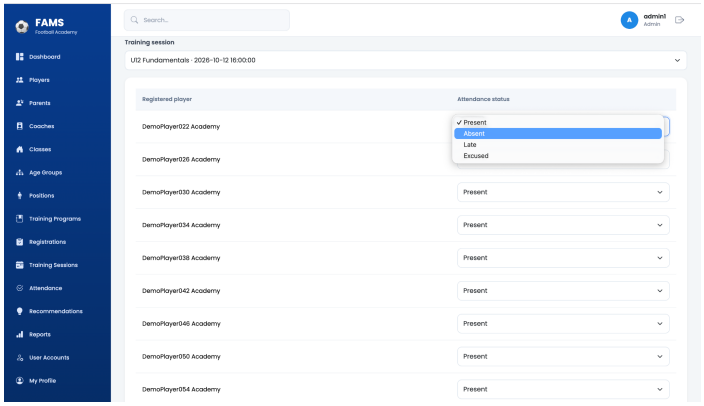


Figure 7. Record training-session attendance.

SQL Reports

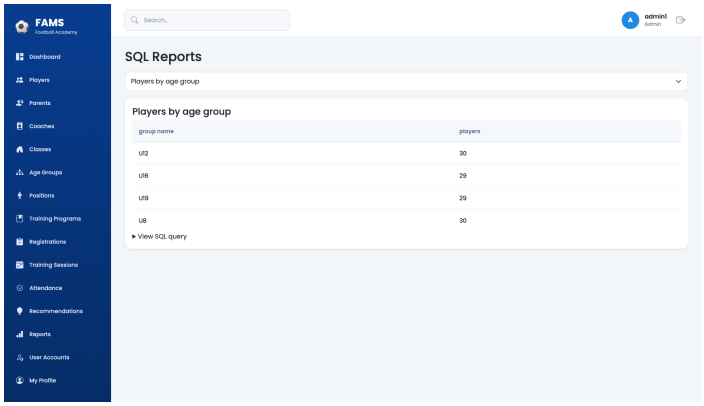


Figure 8. View SQL results by age group.